

Hospital Management System

Sam Schaefer

05/04/2026

Info C-451

Table of Contents

Customer Problem Statements

System Requirements

Functional Requirement Specifications

System Sequence Diagram

Activity Diagram

User Interface Specifications

Implementation Planning

System Architecture and System Design

Algorithms and Data Modeling Implementation

User Interface Design and Implementation

Design of Test

Error Handling and Debugging

Collaboration and Code Integration

Performance Optimization Plan

Maintenance and Support Strategy

Customer Problem Statements

The current healthcare management system relies on a lot of manual processes and outdated record keeping methods that decrease efficiency and limit the amount of accurate and up to date information available to employees. With the new information system, we will look to update many different aspects of the current administration functions of the hospital. From the patient records portal, employee scheduling and staffing, duties such as billing and employee pay. The new system will look to smoothly transition from the old system of record with accurate data and more secure processes that also allow for more ease and automation for employees.

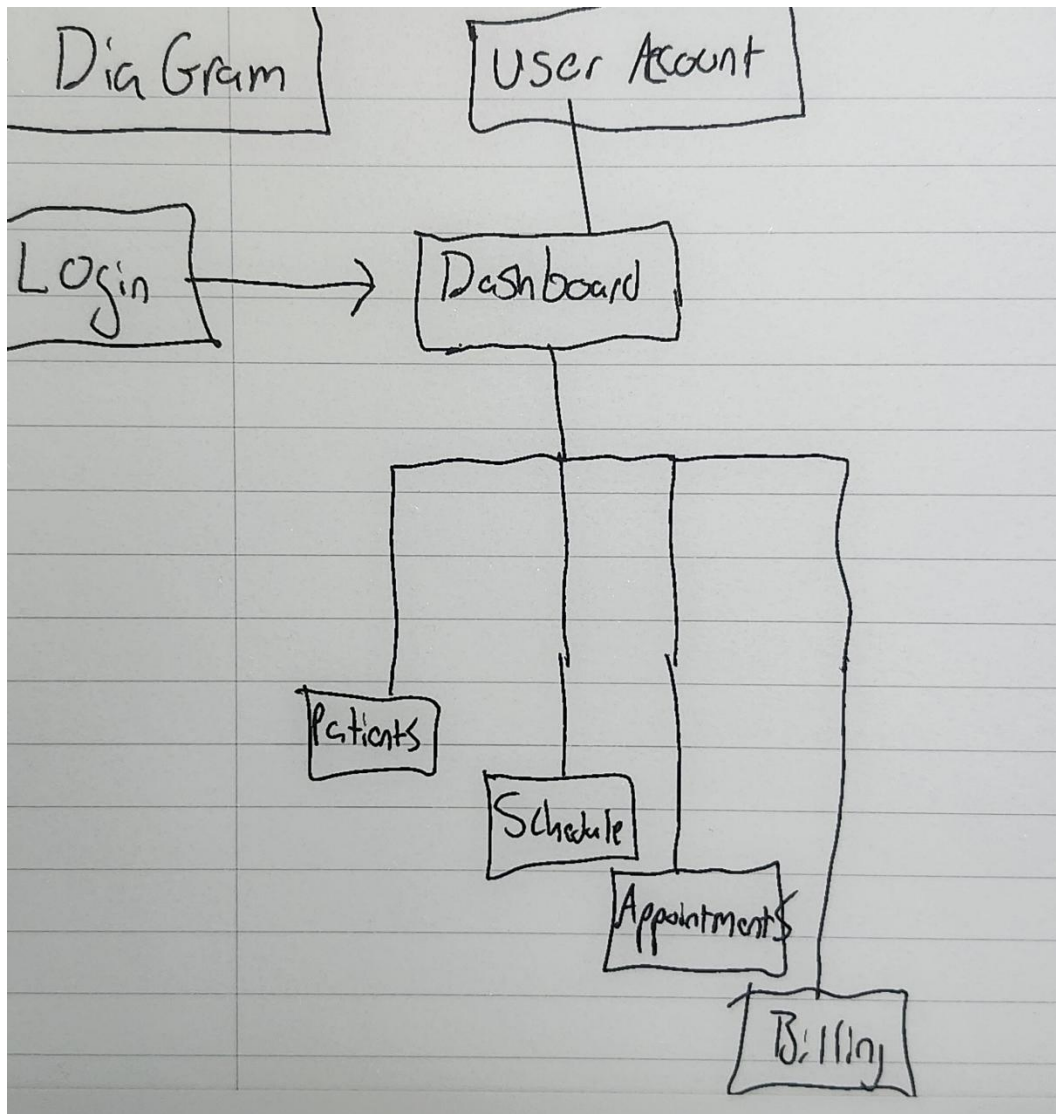
System Requirements

- Latest Windows Operating System
- Java JDK (IntelliJ or Eclipse) with FX installed
- SQLite
- 4GB RAM
- 500MB disk space

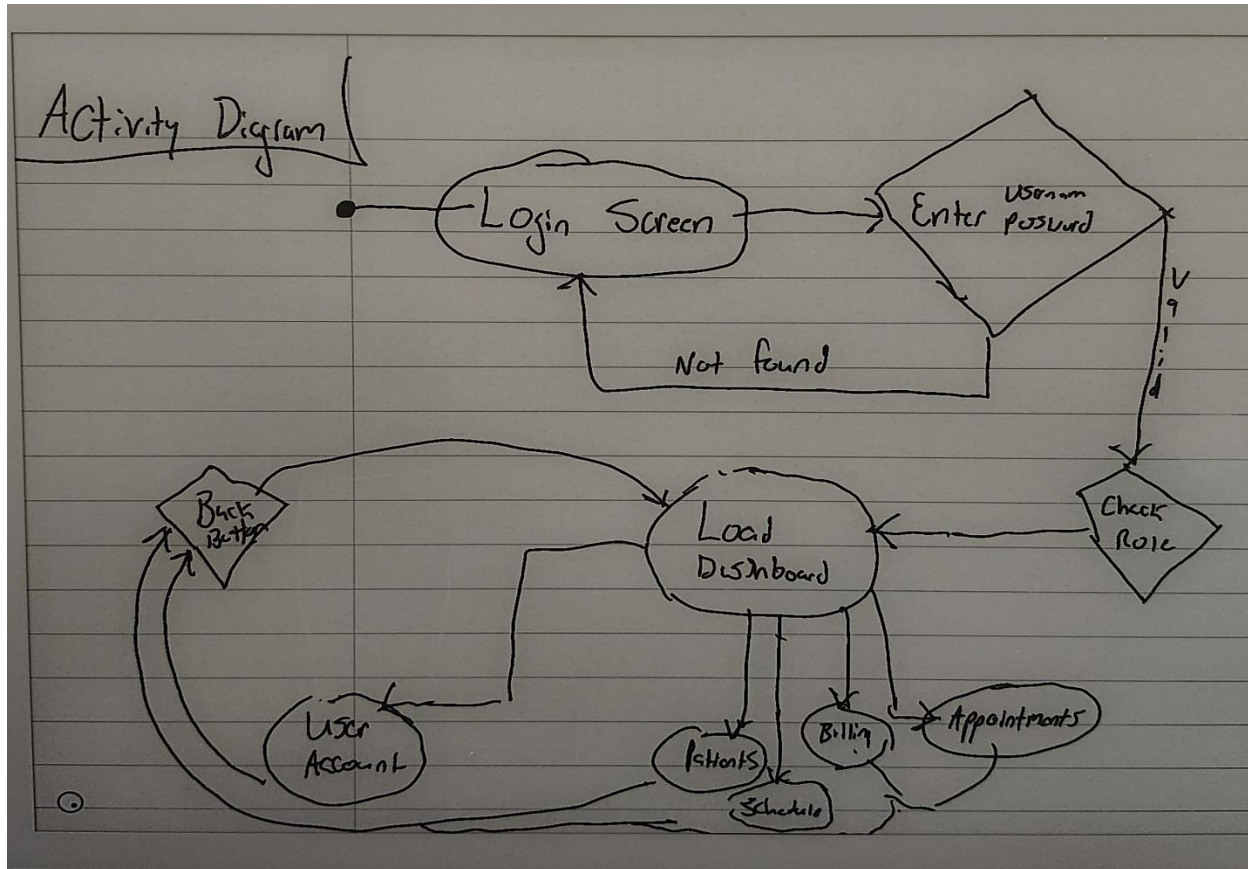
Functional Requirement Specifications

- Role based login system and authentication- based on the different role you are assigned, such as doctor, nurse, patient, or administrator, you will have access to different windows and options. Username and password protected.
- Patient Records, Billing, and Appointment Access- will be able to see patient history, doctor notes, and appointments.
- Employee Schedule- Employee's will be able to access their schedule to see which hours and days they work
- Customizable User Accounts

System Sequence Diagram



Activity Diagram



User Interface Specifications

- Login Page
 - Red and Cream background with shaded lettering in the text fields of each
 - Hospital Emoji logo
 - Blue login button to standout
 - Bright Yellow error notifications for invalid entries
- Dashboard
 - Navy Blue with the same Hospital Logo, but now in black
 - Navigational buttons that change when mouse hovers over them
 - Role specific account button in right corner
- Patient Window
 - Table spreadsheet displaying patient information
 - Return to dashboard button at the bottom
- Employee Window
 - Table spreadsheet with employee information
 - Return to dashboard button at the bottom

- Appointments Window
 - Table spread sheet with appointment information(Patient ID, assigned doctor, reason for visiting, date, and status of the appointment)
 - Color coded system for appointment status (Green is completed, Red cancelled, black is scheduled)
 - Back button to dashboard at the bottom
- Billing Window
 - Table spreadsheet with billing information (PatientID, Amount owed, etc)
 - Color coded status: with red being past due, green being paid, and orange for pending.
 - Back to dashboard button at the bottom
- User Account Page
 - Displays current user
 - Options to change name
 - Back to dashboard button at the bottom

Implementation Planning

Objectives:

- Maintain accuracy and security of old records and patient/employee data
- Improve accuracy and accessibility to patient and employee data
- Increase efficiency of hospital workforce
- Decrease manual workload for employees

Strategy:

- Extensive requirement gathering and old system analysis
- Incremental build of test system based on requirements with constant user feedback
- Testing and user testing with feedback
- Training documentation and walkthroughs for users
- Incremental push to new system
- Continual Support and error corrections

System Architecture and System Design

Layered Architecture:

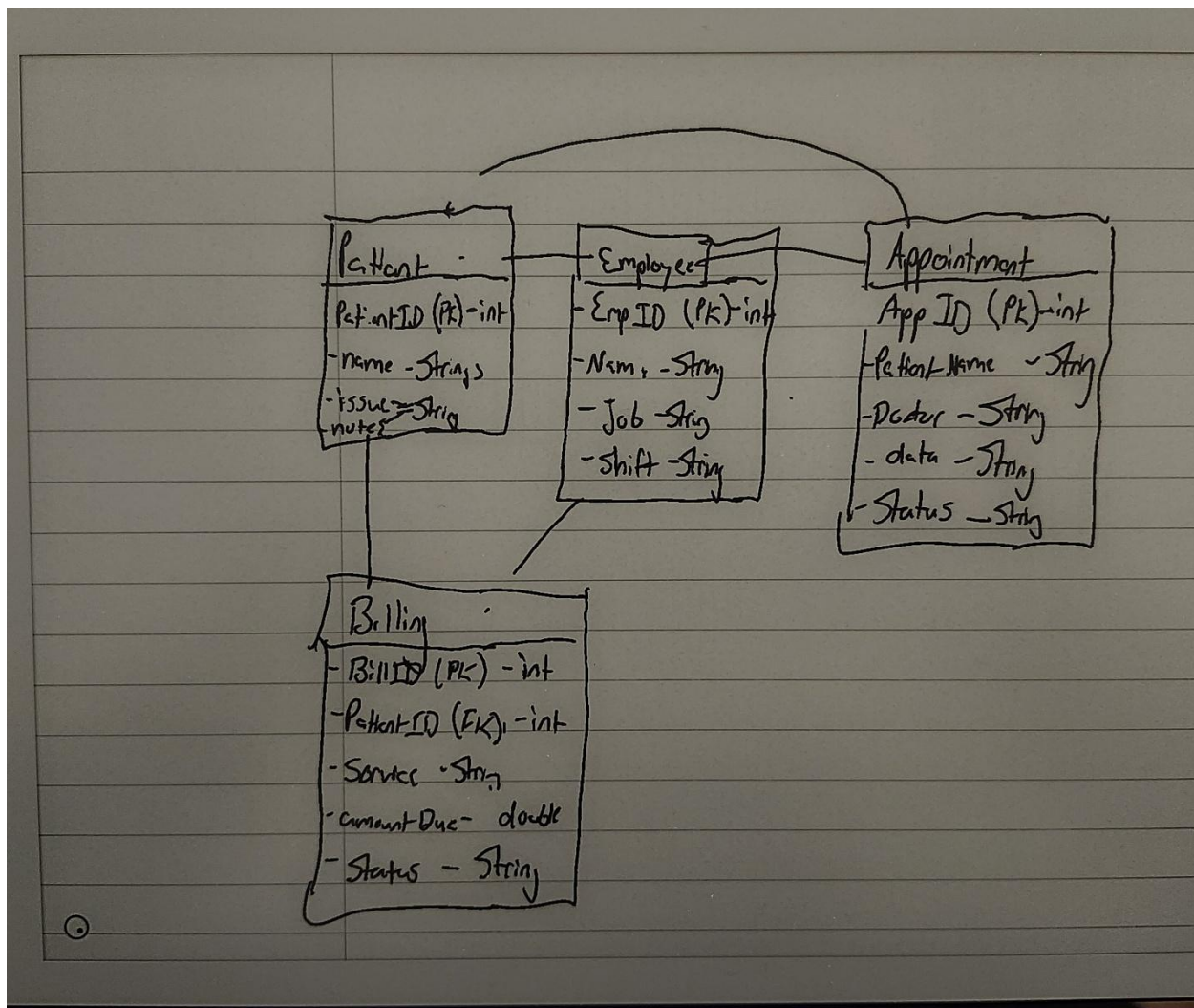
- The frontend UI dashboard with different FX windows
- Backend code in Java with role-based access, input validation, and button navigation.
- Backend database layer on SQLite that handles the saving of user information

System Design:

- Login Page: started by the main class and then login credentials checked by login class and directs to Dashboard
- Dashboard class shows the different navigational buttons available and user button based on the user that is logged in
- Role based access/ authentication: different buttons will be displayed based on what role you have. Admin can see all, employees can see patients and schedule, and patients can only see their appointments
- Each class for individual windows: employee, patient, schedule, appointment, billing, user accounts, etc.
- Database classes handle the connection to SQLite and initialization of the tables
- Logging classes log limited login data
- Test classes with Junit

Algorithms and Data Modeling Implementation

- Algorithms:
 - getRole- helps match role to the specific layout of the screen. Checks if roles are matching and if not goes on to the next if/else until it gets to the least access role, patients
 - updateItem: Changes colors based on whether the text inside the tables matches certain qualifications
- Data Model:



User Interface Design and Implementation

- Used built in tools of the IDE with Java FX to code the different layouts and color schemes.

- Attached a separate CSS stylesheet with certain qualifications to make the code less cumbersome and easier to read.

- Used BorderPane, HBox, and VBox to set the Frame layout

- Login screen displays with different color scheme and different buttons will appear based on role

- Login button will display name of user

Design of Test

-Used built in Junit from Java and sample test code to check that each user and login information was working correctly

-Used SQLite calls to check and see if the database connections were made and tables successfully created

Unit Testing:

- Used built in Junit for Eclipse 3/3
 - Patient
 - Employee
 - Appointment

Integration Testing:

Junit: 4/4 testing

Error Handling and Debugging

- Recreate in test environment to limit scope
- Work top down or bottom up to try to find exactly where the error occurred
- Log all errors and when they occur
- Use automative systems to catch slight errors in large chunks of code that are easily missed by humans
- Check online for the error codes displayed in the console

Collaboration and Code Integration

- Code developed in Eclipse, but house on Github publicly
- Repository contains All java files, CSS Stylesheet code, the SQLite database files, and ReadME files for implementation.
- Development workflow was based on project requirement gathering, brainstorming and then research into how to structure and troubleshoot coding. Started with the main and login page setup first to at least get those

working and then focused on the style on them and the setup of each of the windows. After that it was the buttons and navigation work and algorithms. SQLite connection and integration check came next. The testing code and testing was last with Junit unit testing and integration test and then logging code.

- Tried to keep each page consistent in its layout and style if they were similar in nature.
- Naming conventions made sense and were relevant and consistent with proper comments on each to highlight their use.

Performance Optimization Plan

- Current state of system is a working beta model that is intended to be the preliminary model to give stakeholders a vision of what could be down the road. It is still very limited because of costs and the limitations of project development team.
- The database is very rudimentary and would need to be invested in and expanded to be actually useful. It is not at the moment fully functional. Chose SQLite for ease of use and low cost in integrating with the IDE. Will need further refinement and more specification and normalization of the tables and integration. Database could probably also benefit from indexing, especially if there will be more people using the system.
- UI is also pretty basic and could use more refinement to not only make it more interesting, but to also add more customer features and ease of use. Would like to speed up the load times and make the functionality a little less cumbersome

Maintenance and Support Strategy

- Employee persona check and validation post go live and continued support
- Continued persona testing and validation on select profiles based on known errors or irregularities
- Compare data with old system during different periods of time
- Ongoing updates to help increase efficiency and align with current technology.